Experiment - 8

Student Name: Shresth Garg
UID: 23BCS11679
Branch: BE-CSE
Section/Group: KRG-2A
Semester: 6th
Date of Performance: 08/04/2026
Subject Name: System Design
Subject Code: 23CSH-314

Aim: To design a **stock-trading platform (Similar to Zerodha, Groww & Upstock)** These platforms serve as the commission-free online platform for trading and monitoring of the stocks.

Objectives:

- To understand the architecture of real-time stock trading systems.
- To identify functional requirements such as trading, portfolio management, and watchlists.
- To define non-functional requirements including scalability, latency, and consistency.
- To design APIs for user authentication, trading operations, and portfolio management.
- To analyze real-time data processing using streaming systems like WebSockets and Kafka.

Procedure:

1. Studied existing stock trading platforms such as Zerodha, Groww, and Upstox.
2. Identified core entities like Users, Stocks, Orders, Trades, Portfolio, and Watchlist.
3. Collected functional requirements including trading, real-time data, and portfolio tracking.
4. Defined non-functional requirements such as latency, scalability, and consistency.
5. Designed REST APIs for authentication, trading, funds, and portfolio services.
6. Analyzed real-time stock price updates using WebSockets and Pub-Sub systems.
7. Designed order processing using message queues (Kafka) for reliability and scalability.
8. Created a high-level system architecture integrating all services.

Functional Requirements:

1. Users should be able to do the registration, login and KYC verification.
2. Application should allow users to view real-time stocks prices & its historical data.
3. Application should support placing, modifying, and cancelling orders along with limiting orders with accurate status updates.
4. Application should provide users with a watchlist with real-time updates.
5. Application should provide a dashboard to track current holdings, trade history, P & L, and performance insights.

Core Entities of the System

- Users
- Stocks
- Orders
- Trade
- Portfolio
- Watchlist

API Design

Users:

1. POST: / auth / signup
2. POST: / auth / verify-KYC
3. POST: / auth / login

Market Data:

1. [Listing of all the stocks] WS: / api / v1 / stocks
2. [stock details] GET: / api / v1 / stocks / (stock_symbol_id)
3. [Historical Data] GET: / api/ v1 / stocks / {stock_symbol_id} / history

Funds:

1. POST: / api / v1 / funds / deposit
2. POST: / api / v1 / funds / withdraw
3. GET: / api / v1 / funds / history

Trading:

1. [Place buy / sell order] POST: / api / v1 / orders
2. [List user orders] GET: / api / v1 / {user_id} / orders
3. [Order Status] GET: / api / v1 / orders / {order_id}
4. [Cancel order] DELETE: / api / v1 / orders / [order_id]

Portfolio:

1. GET: / api / v1 / portfolio
2. GET: / api / v1 / portfolio / holdings
3. GET: / api / v1 / portfolio / positions
4. GET: / api . v1 / portfolio / performance [OPTIONAL]

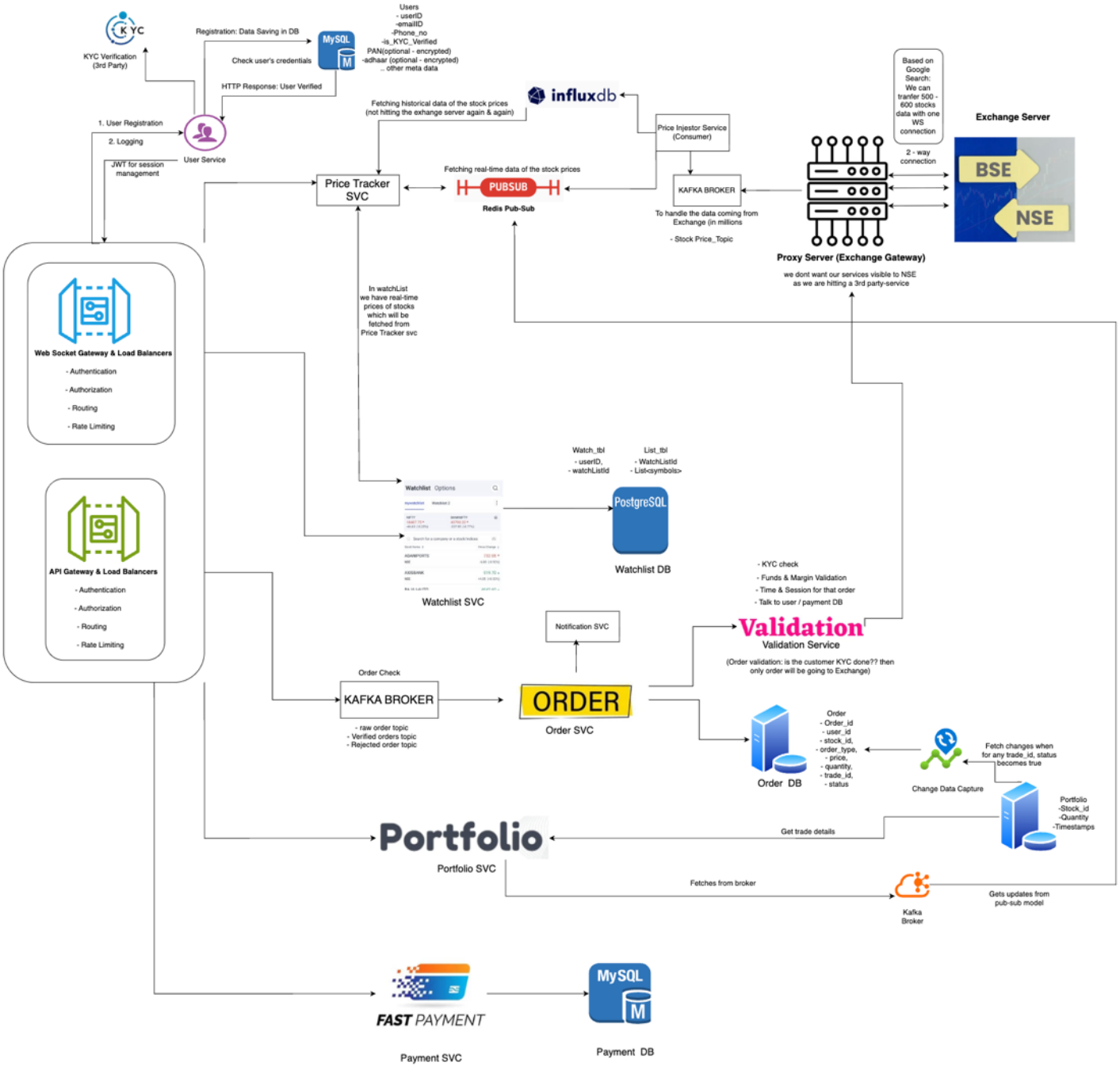
Watchlist:

1. GET: / api / v1 / watchlists
2. POST / api / v1 / watchlists / {watchlist_id} / symbols

Non-functional Requirements:

1. Scale: 2 - 3M DAU with high-frequency trades, along with 8 - 10k stocks.
2. CAP Theorem: Consistency >>> Availability
   - Correctness of stocks is more important than uptime. (A wrong balance, duplicate trade, or stale price can lead to huge financial losses)
   - Getting the stocks prices in real-time (highliy available)
3. Latency: Order placement < 100ms, market data (updated stock value) < 50ms

High Level Design (HLD):



The system consists of client applications connected through API Gateway and WebSocket Gateway. Market data is streamed in real-time using Kafka and Redis Pub-Sub. Core services include User Service, Order Service, Portfolio Service, Watchlist Service, and Payment Service. Orders are validated and processed asynchronously, while databases like MySQL and PostgreSQL store transactional and user data.

Learning Outcomes:

- Designed a scalable ride-sharing system with real-time capabilities.
- Learned how ride booking and driver matching systems work.
- Understood trade-offs between consistency and availability.
- Implemented structured API design for rider and driver workflows.
- Learned about real-time systems using WebSockets and geo-indexing.